OPERATING SYSTEMS PROJECT DOCUMENTATION

SJF (Preemptive) vs. Priority (Preemptive)

SENARIO A: Basic Mixed workload

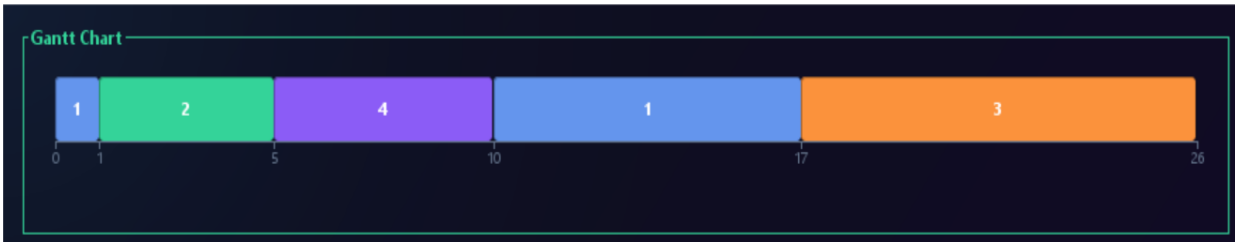
Current Processes List

Process Table

Process ID	Arrival Time	Burst Time	Priority
1	0	8	3
2	1	4	1
3	2	9	4
4	3	5	2

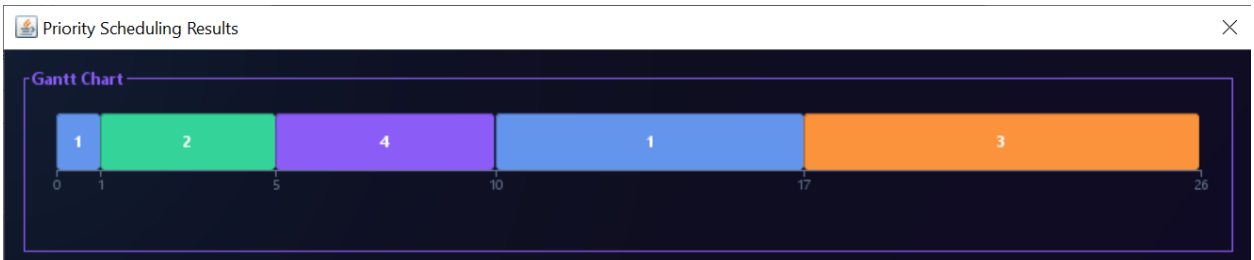
Preemptive SJF metrics Result (Wt,Rt& Tat)

SRTF (Preemptive SJF) Results



Process ID	Arrival	Burst	Waiting	Turnaround	Response
1	0	8	9	17	0
2	1	4	0	4	0
3	2	9	15	24	15
4	3	5	2	7	2
Average	—	—	6.50	13.00	4.25

Preemptive Priority metrics Result (Wt,Rt& Tat)



Process ID	Arrival	Burst	Priority	Waiting	Turnaround	Response
1	0	8	3	9	17	0
2	1	4	1	0	4	0
3	2	9	4	15	24	15
4	3	5	2	2	7	2
Average	—	—	—	6.50	13.00	4.25

Analysis Questions

- Which algorithm gave lower average waiting time? Both have equal average waiting time (6.50)
- Which algorithm gave lower average turnaround time? Both have equal average turnaround time (13.00)
- Did SJF favor short jobs more strongly? Yes, SJF (SRTF) prioritized shorter jobs, specifically P2 and P4.
- Did Priority Scheduling favor urgent processes more strongly? Yes, it prioritized processes with the lowest priority values (P2 then P4), ensuring that urgent tasks ere completed first.
- Was any starvation or unfair delay observed? No severe starvation was observed, but P3 (the longest and lowest priority job) experienced the longest delay, waiting 15 units before starting.
- Which algorithm would you recommend for the tested workload, and why? SRTF (Preemptive SJF) is recommended to minimize average wait times, while Priority is better if tasks have priorities regardless of their size.

- Required Conclusion

- Algorithm Performance: Both SRTF and Preemptive Priority performed identically in Scenario A. This occurred because the shortest jobs were also the ones with the highest priority.
- Efficiency vs. Urgency Trade-off: Efficiency (SRTF) focuses on system throughput by clearing short jobs quickly. Urgency (Priority) ensures critical tasks are completed first, even if it reduces overall system speed.
- Fairness in Practice: SRTF is generally fairer as it reduces wait times for the majority of processes. Priority can be "unfair" to low-priority tasks, potentially causing starvation (as seen with Process 3 waiting 15 units).

Scenario B: Conflict between Burst Time and Priority

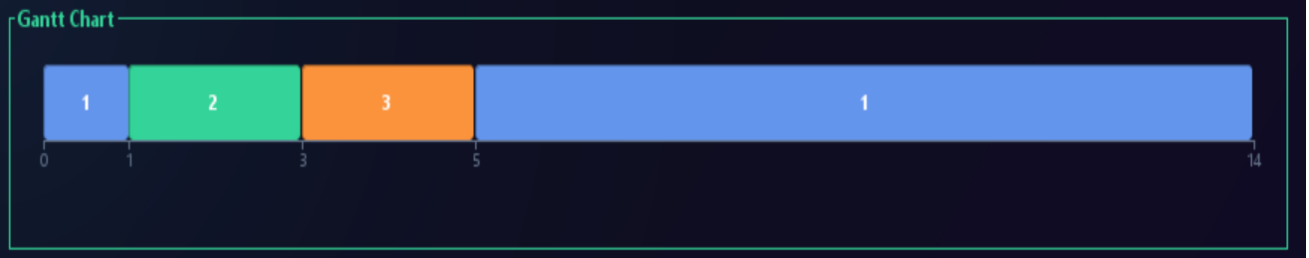
Current Process list

Process Table

Process ID	Arrival Time	Burst Time	Priority
1	0	10	1
2	1	2	5
3	2	2	4

Preemptive SJF metrics Result (Wt,Rt& Tat)

SRTF (Preemptive SJF) Results



Process ID	Arrival	Burst	Waiting	Turnaround	Response
1	0	10	4	14	0
2	1	2	0	2	0
3	2	2	1	3	1
Average	—	—	1.67	6.33	0.33

Preemptive Priority metrics Result (Wt,Rt& Tat)

Process ID	Arrival	Burst	Priority	Waiting	Turnaround	Response
1	0	10	1	0	10	0
2	1	2	5	11	13	11
3	2	2	4	8	10	8
Average	—	—	—	6.33	11.00	6.33

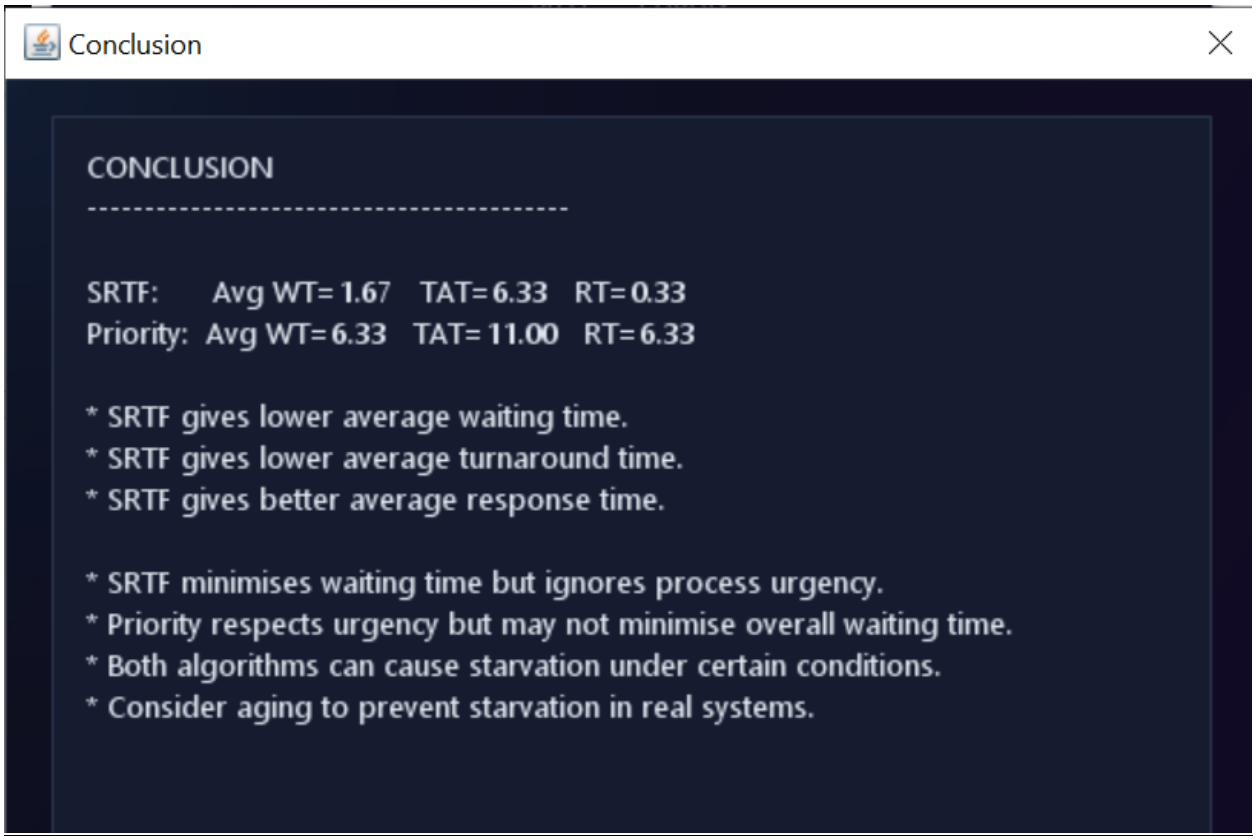
Analysis Questions

- Which algorithm gave lower average waiting time? SRTF provided a significantly lower average waiting time (1.67) compared to Priority (6.33).
- Which algorithm gave lower average turnaround time? SRTF performed better with an average turnaround time of 6.33.
- Did SJF favor short jobs more strongly? Yes, SRTF preempted the long P1 at time 1 to finish the shorter P2 and P3 first.
- Did Priority Scheduling favor urgent processes more strongly? Yes, Priority Scheduling allowed the urgent P1 to complete its full execution first without any interruption.
- Was any starvation or unfair delay observed? Yes, in Priority Scheduling, the short jobs (P2 and P3) suffered an unfair delay, waiting until the long P1 finished

- Which algorithm would you recommend for the tested workload, and why? Priority is recommended if P1 is a critical system task, but SRTF is recommended for general efficiency and faster response.

Required Conclusion

- Algorithm Performance: SRTF performed better in terms of numerical metrics (Wait/Turnaround), while Priority performed better in respecting task importance.
- Efficiency vs. Urgency Trade-off: The trade-off is between minimizing overall system delay (SRTF) versus ensuring high-priority tasks finish as soon as possible (Priority).
- Fairness in Practice: SRTF appeared fairer to the majority of processes, whereas Priority was "unfair" to shorter tasks with lower importance.



Scenario C: Fairness or Starvation-Sensitive Case

Current Process list

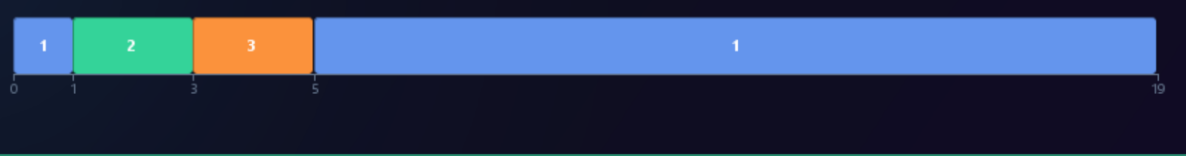
Process Table

Process ID	Arrival Time	Burst Time	Priority
1	0	15	10
2	1	2	1
3	2	2	2

Preemptive SJF metrics Result (Wt,Rt& Tat)

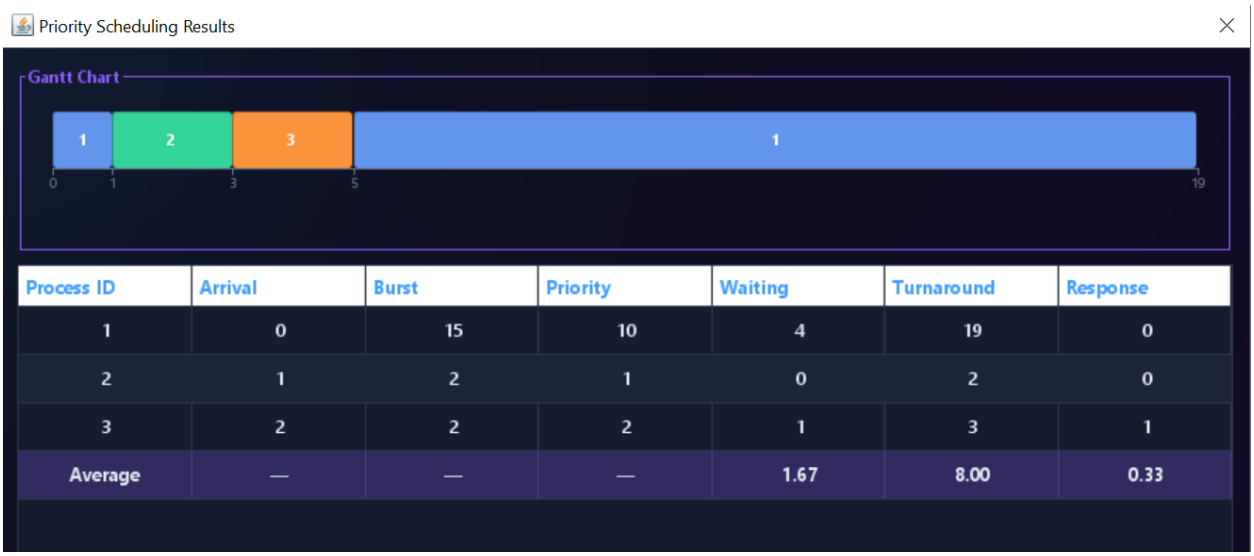
SRTF (Preemptive SJF) Results

Gantt Chart



Process ID	Arrival	Burst	Waiting	Turnaround	Response
1	0	15	4	19	0
2	1	2	0	2	0
3	2	2	1	3	1
Average	—	—	1.67	8.00	0.33

Preemptive Priority metrics Result (Wt,Rt& Tat):



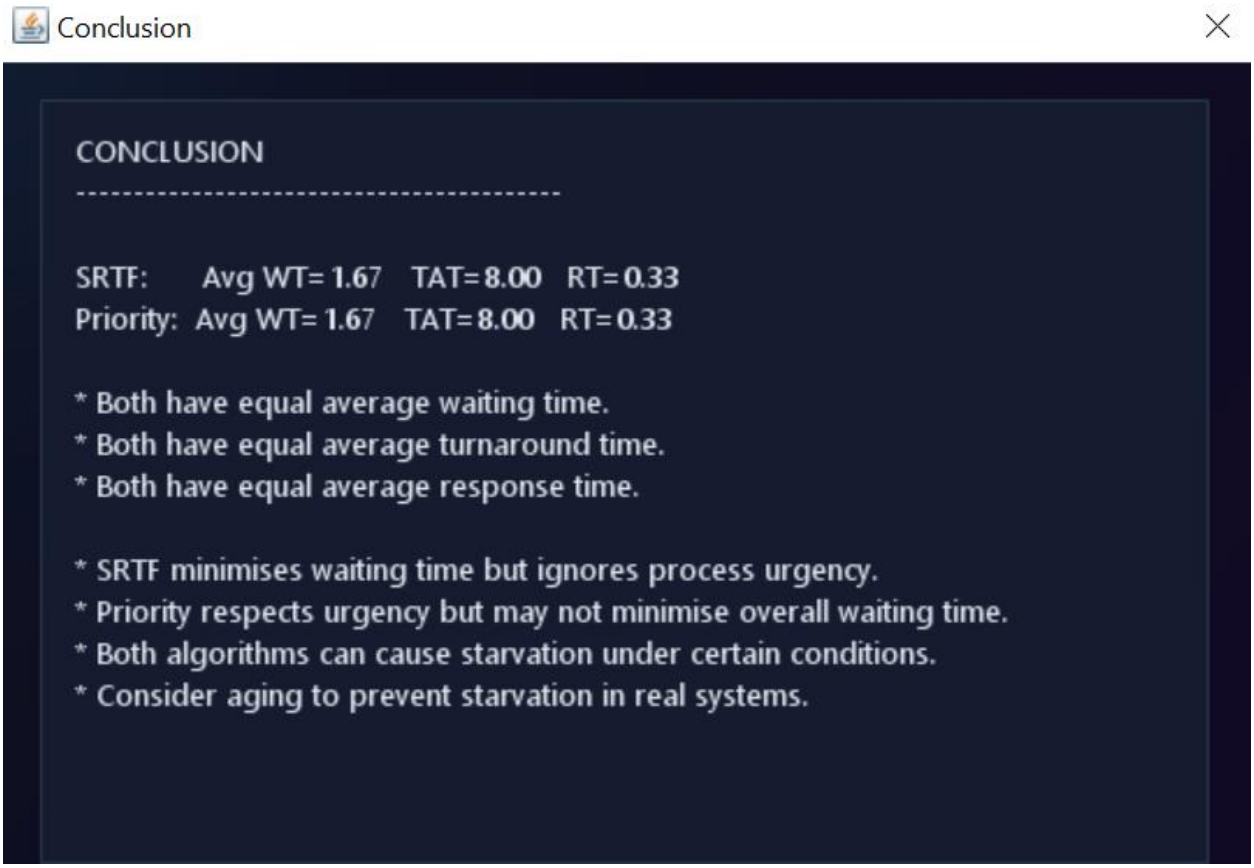
Process ID	Arrival	Burst	Priority	Waiting	Turnaround	Response
1	0	15	10	4	19	0
2	1	2	1	0	2	0
3	2	2	2	1	3	1
Average	—	—	—	1.67	8.00	0.33

Analysis Questions

- Which algorithm gave lower average waiting time? Both algorithms resulted in the same average waiting time (1.67) due to the identical execution order.
- Which algorithm gave lower average turnaround time? Both yielded an average turnaround time of (8.00).
- Did SJF favor short jobs more strongly? Yes, SRTF preempted P1 at time 1 because P2 and P3 had much shorter burst times.
- Did Priority Scheduling favor urgent processes more strongly? Yes, it prioritized P2 and P3 due to their higher priority (1 and 2), forcing P1 to wait
- Was any starvation or unfair delay observed? Yes, P1 experienced a significant unfair delay. While it arrived first, it was forced to wait for all other processes to finish. If more short or high-priority processes kept arriving, P1 would suffer from Total Starvation.
- Which algorithm would you recommend for the tested workload, and why? SRTF is recommended for general workloads to keep the system responsive, but Aging techniques should be added to either algorithm to prevent P1 from waiting forever.

Required Conclusion

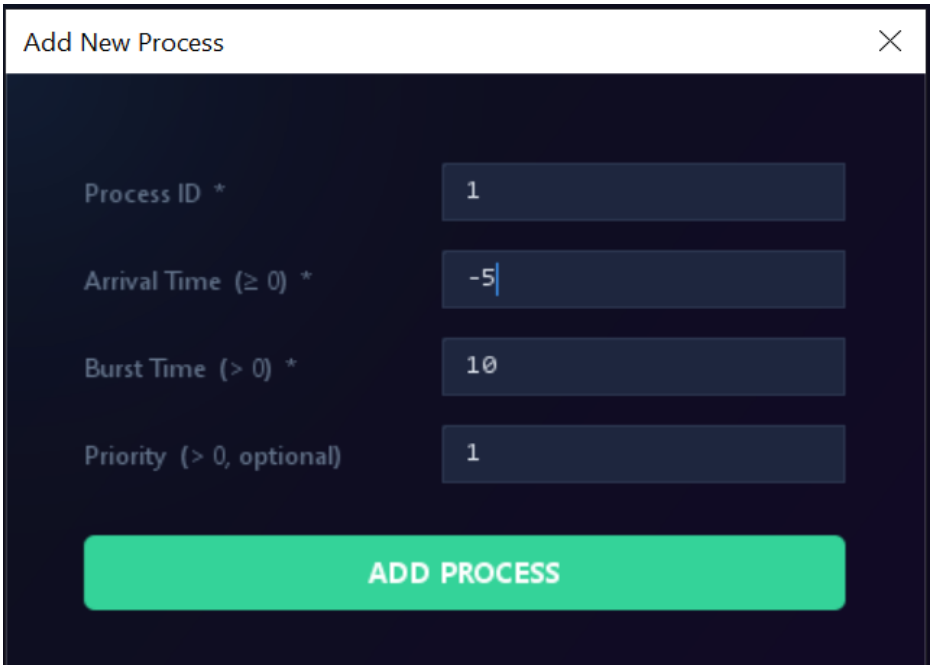
- Algorithm Performance: Both algorithms handled the workload similarly, but they both exposed P1 to potential starvation.
- Trade-off: The trade-off is between maximizing throughput (clearing many small jobs) versus maintaining fairness for long or low-priority tasks.
- Fairness in Practice: Neither algorithm appeared "fair" to P1. In practice, SRTF is usually fairer because it is based on task size rather than potentially biased priority values.

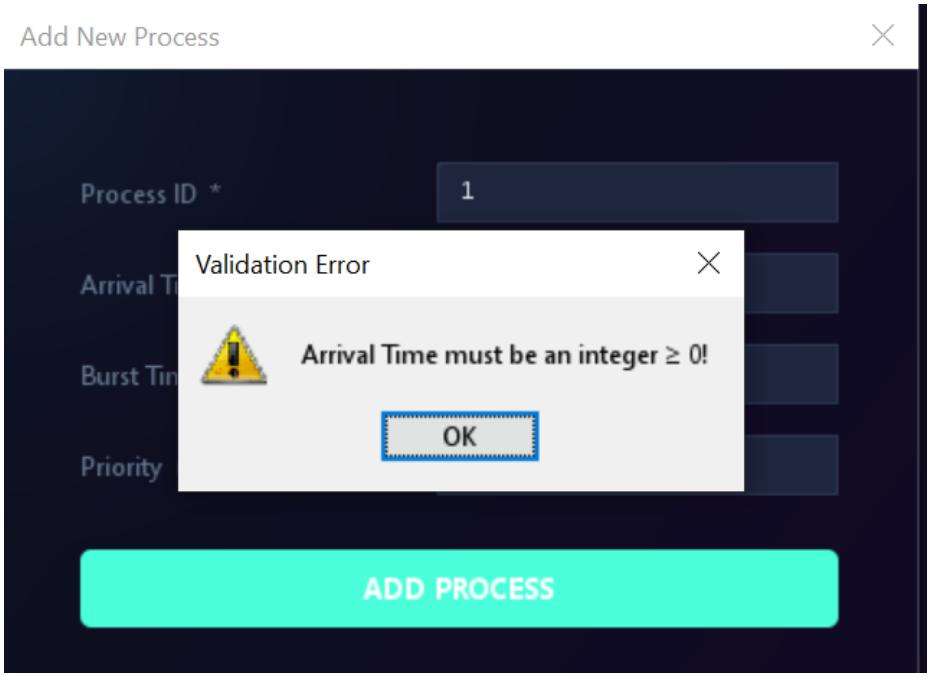


Scenario D: Validation Case

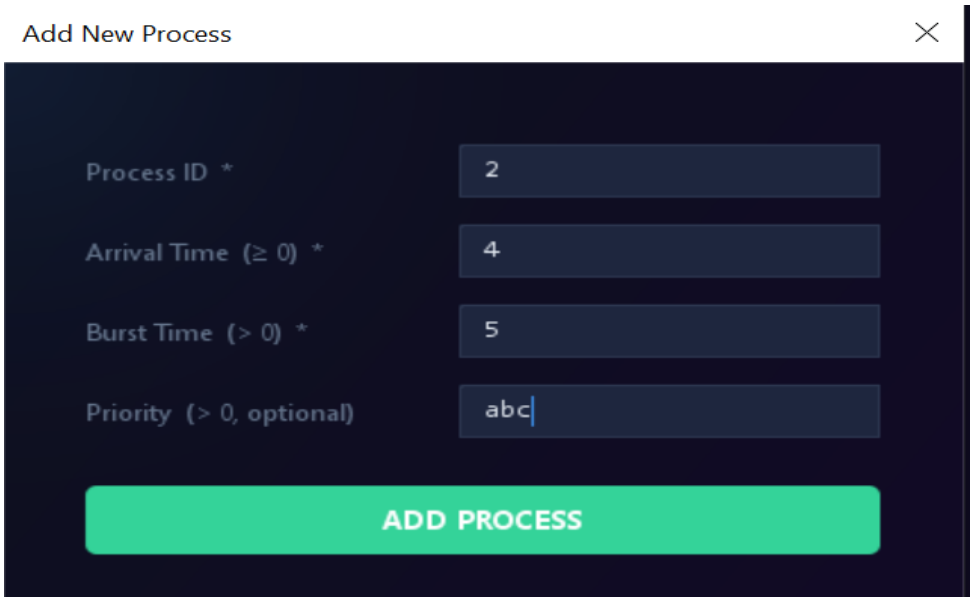
This scenario demonstrate the system's ability to handle invalid inputs and prevent logical errors in scheduling.

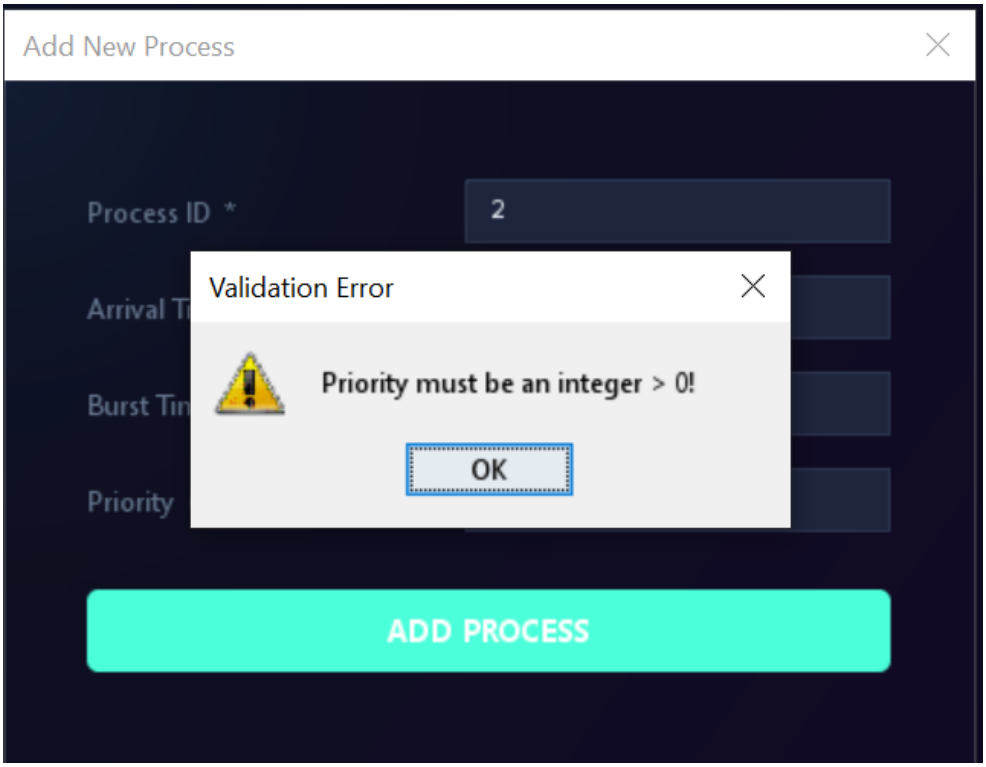
Case 1: When inserting arrival time value by -5 (arrival time cant be-ve number), validation error message appear to the user as shown below.





Case 2: When inserting priority value by abc (priority value must be a + ve number), validation error message appear to the user as shown below:





Required Conclusion

- Metric Handling: The system handled these cases by rejecting them to maintain Data Accuracy.
- Trade-off: The trade-off is between User Flexibility and System Stability. Strict validation ensures stability even if it requires the user to re-enter data.
- Fairness in Practice: Validation is the fairest approach because it ensures the scheduler doesn't produce misleading results based on faulty data

THANKS :)